GAME CLIENT PORTFOLIO

클라이언트 프로그래머

박신우 Park Shin Woo

E-Mail: tlsdn0403@gmail.com

GitHub: github.com/tlsdn0403

PROJECT INDEX

02

주요 프로젝트

01 / GRADUATION PROJECT

감염: 죽음의 도시

Unreal Engine 5 · C++

Co-op TPS · Zombie Survival

프로젝트 보기 ↗

02 / RENDERING STUDY

Shader Lab

OpenGL · GLSL · C++

라이팅, 셰이더, GI와 LOD 실험

프로젝트 보기 ↗

03 / NETWORK GAME

Fall Guys Network

C++ · OpenGL · WinSock

TCP/IP 멀티플레이

프로젝트 보기 ↗

04 / AI VIBE CODING

FlickDom

Unity 6 · C# · MCP

1 vs 1 전략 플릭 보드게임

프로젝트 보기 ↗

함께 소개된 프로젝트 · 버블로그 원본 포트폴리오 목차에 수록된 2D 액션 로그라이크

원본 목차 보기 ↗

PROJECT 01 / PAGE 01

감염: 죽음의 도시

기간	2025.12.20 - 2026.07.08
개발인원	3인팀
엔진	Unreal Engine 5 · C++
장르	Co-op TPS · Zombie Survival
작품 설명	2025.12.20 - 2026.07.08

팀원과의 소통방법

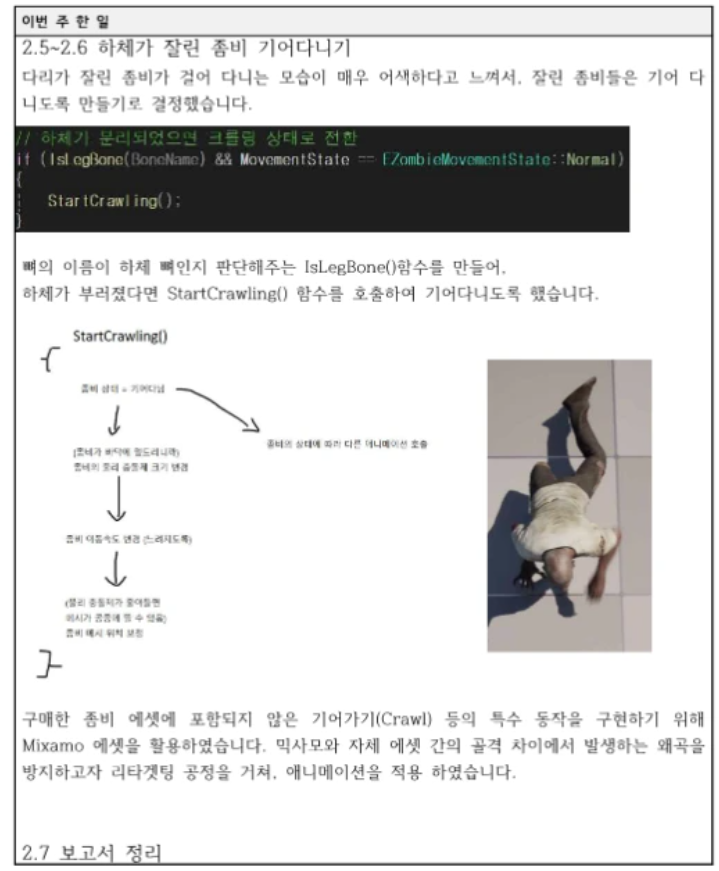
조장으로서 개발 일정을 함께 짜고, 보고서를 매주 전송하며, 서로의 진행상황을 공유하였습니다.

계획서 작성

| 06 개발일정

박신우 : ○, 배주환 : ○, 안윤진 : ○, 공동 : ○	1월	2월	3월	4월	5월	6월	7월	8월
리소스 수집								
기본 캐릭터 기능								
차량 기능								
차량 충돌 기능								
차량 충기 기능								
차량 유탄 기관총 폭발 효과								
차량 연기 이펙트 구현								
차/좀비 충돌 시 피튀기는 효과								
좀비 애니메이션								
좀비 기능								
좀비 AI 행동 트리								
서버 프레임워크								
멀티 스레드 병렬처리								
서버 내 충돌처리 로직								
좀비 AI 관련 서버 처리								
맵 디자인								
화면 UI								
사운드, 게임 플레이								
테스트 및 버그수정								

보고서를 통한 진행상황 보고



이번 주 한 일

2.5~2.6 하체가 잘린 좀비 기어다니기

다리가 잘린 좀비가 걸어 다니는 모습이 매우 어색하다고 느껴서, 잘린 좀비들은 기어 다니도록 만들기로 결정했습니다.

```
// 하체가 분리되었으면 크롤링 상태로 전환
if (IsLegBone(BoneName) && MovementState == EZombieMovementState::Normal)
{
    StartCrawling();
}
```

뼈의 이름이 하체 뼈인지 판단해주는 IsLegBone()함수를 만들어,
하체가 부러졌다면 StartCrawling() 함수를 호출하여 기어다니도록 했습니다.

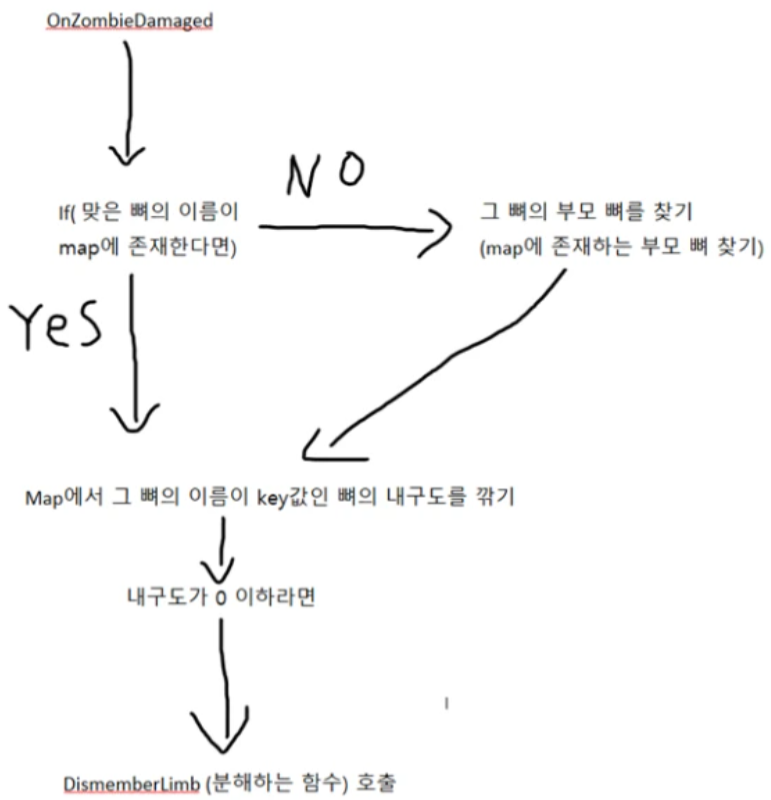
구매한 좀비 에셋에 포함되지 않은 기어가기(Crawl) 등의 특수 동작을 구현하기 위해 Mixamo 에셋을 활용하였습니다. 믹사모와 자체 에셋 간의 골격 차이에서 발생하는 왜곡을 방지하고자 리타겟팅 공정을 거쳐, 애니메이션을 적용 하였습니다.

2.7 보고서 정리

이 프로젝트에서 맡은 역할

좀비 신체 분리

구조 설계



코드 작성

ABaseZombie::OnZombieDamaged

```
void ABaseZombie::OnZombieDamaged(
    float Damage,
    FName HitBoneName,
    FVector Impulse,
    FVector HitLocation
)
{
    if (HitBoneName.IsNone() || BrokenBones.Contains(HitBoneName))
        return;

  // 피격된 뼈가 직접 등록되어 있지 않으면 부모 뼈에서 분리 대상 검색
    FName DamageBoneName = FindDismemberTargetBone(HitBoneName);

    if (DamageBoneName.IsNone())
        return;

    int32* BoneDurability = BoneDurabilityMap.Find(DamageBoneName);

    if (BoneDurability == nullptr)
        return;

    *BoneDurability -= FMath::RoundToInt(Damage);

    if (*BoneDurability <= 0)
    {
        DismemberLimb(DamageBoneName, Impulse, HitLocation);
    }
}
```

이 프로젝트에서 맡은 역할

좀비 신체 분리

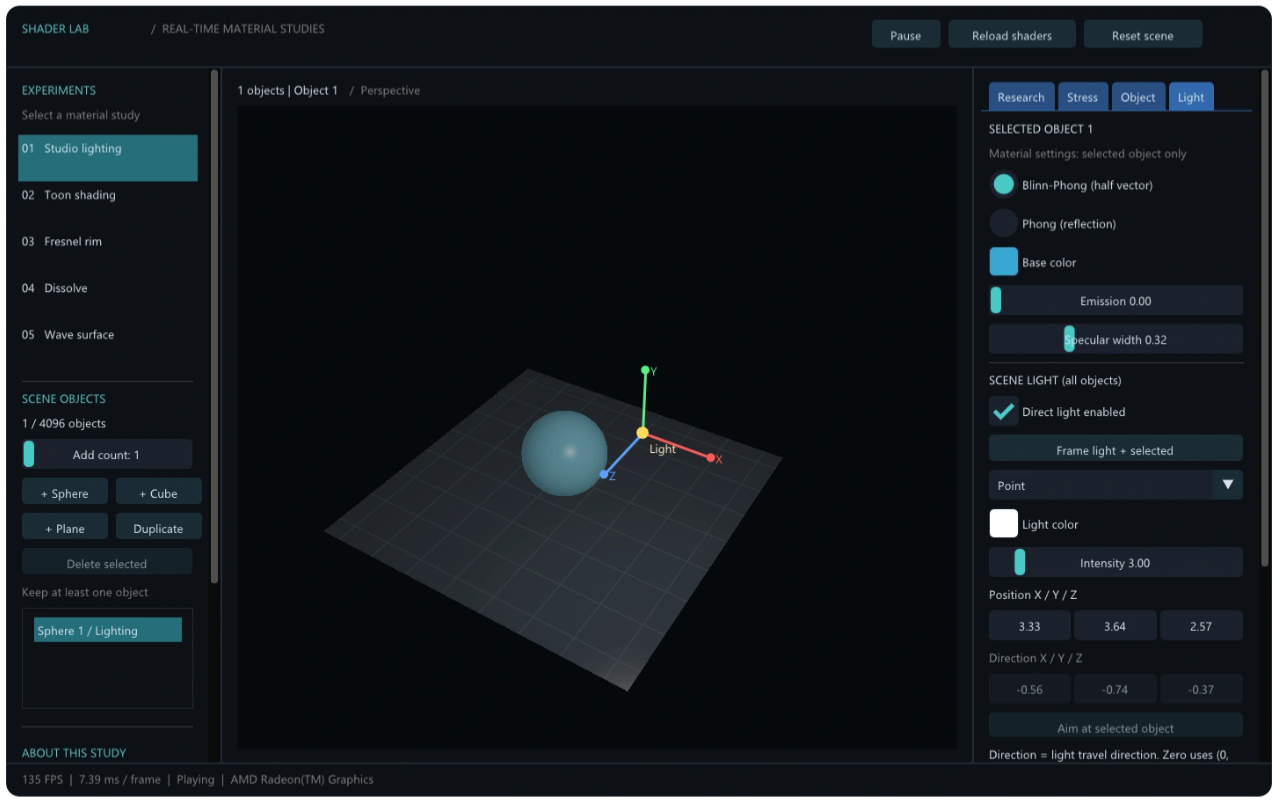
하체 분리 → 기어가기

이 프로젝트에서 맡은 역할

트럭 운전자 시점

트럭 기관총 사용자 시점

Shader Lab



실제 실행 화면 · 광원과 재질을 조절하는 실시간 편집기

RENDERING PORTFOLIO · 개발 중

직접 조절하고,
차이를 확인하는 셰이더 실험실

셰이더의 수식과 그래픽스 파이프라인을 이해하기 위해 시작한 학습 프로젝트입니다. 직접 작성한 셰이더의 파라미터를 바꾸고 결과를 비교하며, 코드가 화면에 어떤 변화를 만드는지 확인하는 것을 목표로 했습니다.

언어 / 그래픽 API	C++17 · OpenGL 3.3 · GLSL 330
편집기 / 라이브러리	Dear ImGui · GLFW · GLEW · meshoptimizer
핵심 내용	6개 셰이더 · 광원 편집 · Voxel GI · LOD / 컬링
진행 방식	셰이더 직접 학습·작성 / UI·Lab 구조 AI 활용

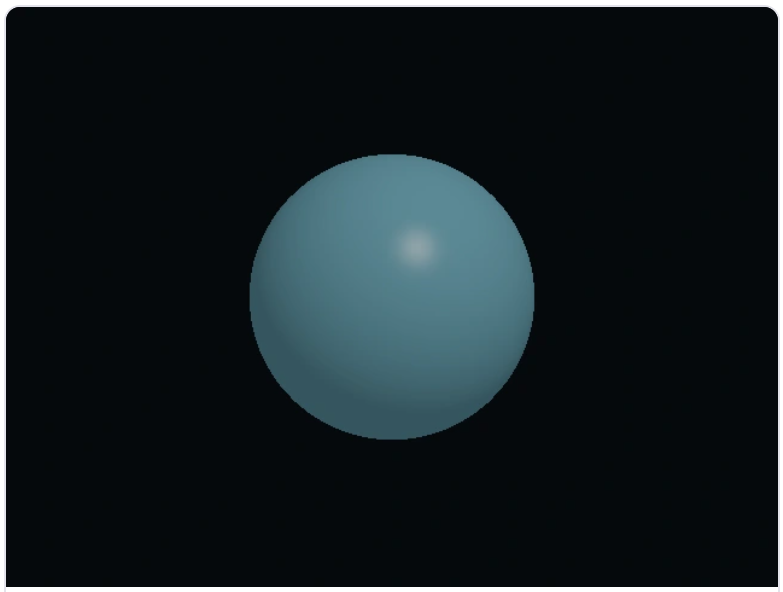
개발 목적과 AI 활용 범위

라이팅과 표면 표현을 공부하며 셰이더 코드를 직접 작성했습니다. 수식을 적용하는 데서 나아가, 값과 계산 방식을 바꿨을 때의 차이를 실행 화면으로 확인할 수 있는 학습 환경이 필요했습니다.

제한된 개발 시간을 셰이더 학습에 집중하기 위해, UI와 전반적인 Lab 구조 및 실행 환경은 AI를 활용해 제작했습니다. 이를 바탕으로 셰이더를 수정하고 실험하며 그래픽스 원리를 익히는 방식으로 진행했습니다.

GitHub 준비 중 | YouTube 준비 중 | Download 준비 중

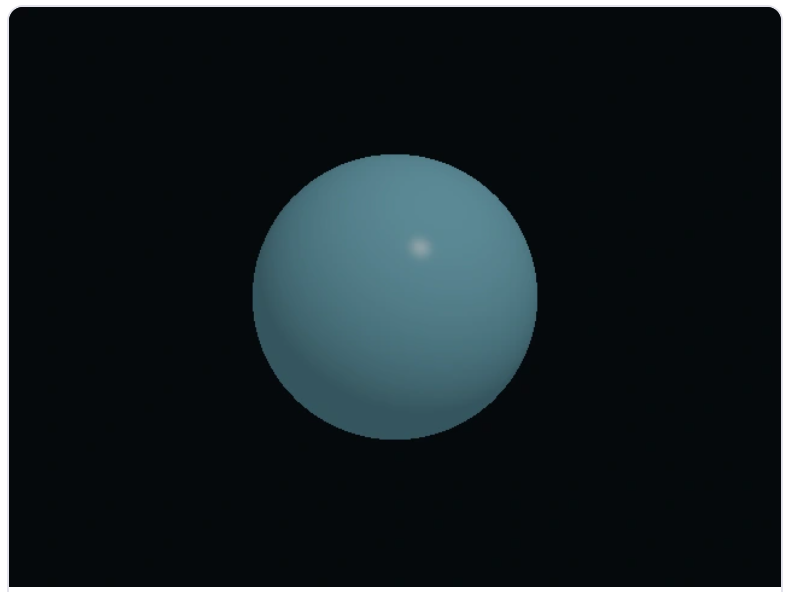
빛과 표면을 표현하는 셰이더



01 · BLINN–PHONG

하프 벡터로 계산하는 하이라이트

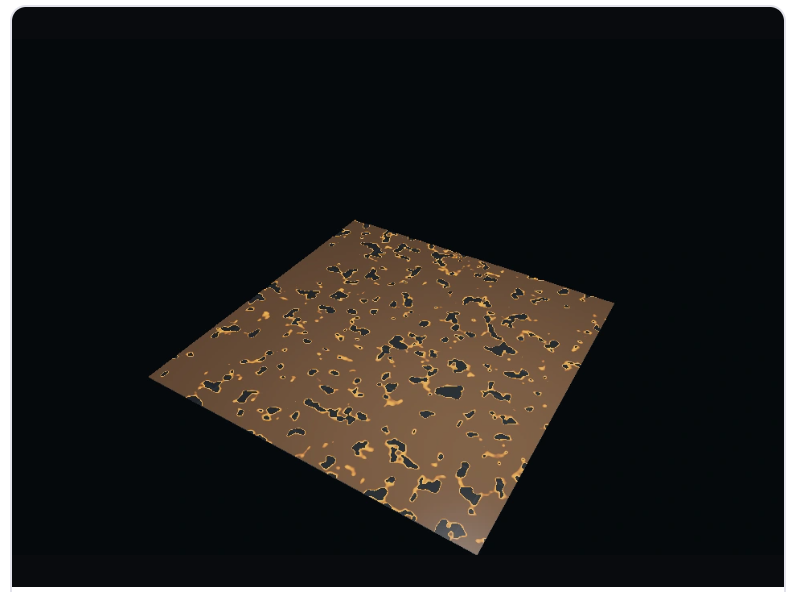
빛 방향과 시선 방향의 중간 벡터 H를 사용합니다. N·H와 지수 값으로 반사광의 범위를 조절합니다.



02 · PHONG

반사 벡터와 시선의 관계

R = 2(N·L)N − L로 반사 방향을 구하고, R·V를 사용해 정반사 성분을 계산합니다.



03 · DISSOLVE

프래그먼트를 버리는 소멸 효과

노이즈와 임계값을 비교해 discard합니다. 구멍처럼 보이지만 이 효과가 정점이나 메시를 삭제하지는 않습니다.

Toon

조명 밝기를 단계화

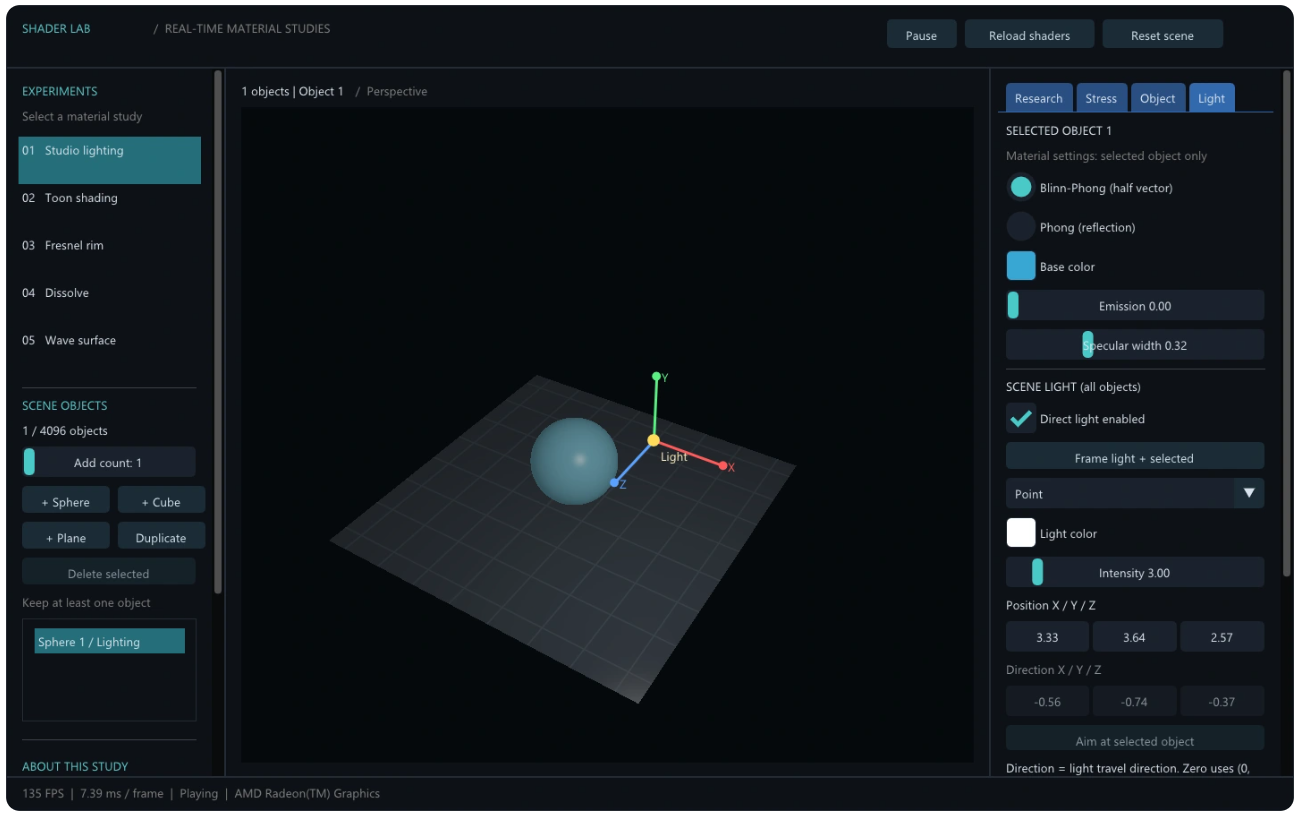
Fresnel

시선 각도에 따른 가장자리 강조

Wave

정점 변위와 법선 보정

Light 탭에서 조절하는 광원



XYZ 기즈모를 드래그해 광원의 위치를 변경

광원 설정은 장면 전체에 적용 · 재질 설정은 선택 객체에 적용 · 기즈모 조작 취소 및 광원 프레이밍 지원

01 / LIGHT TYPE

Directional · Point · Spot

방향광, 점광원, 스폿 광원을 선택합니다. 위치와 방향, 범위, 스폿 각도를 조절해 감쇠의 차이를 비교합니다.

02 / LIVE PARAMETERS

한곳에 모은 라이팅 설정

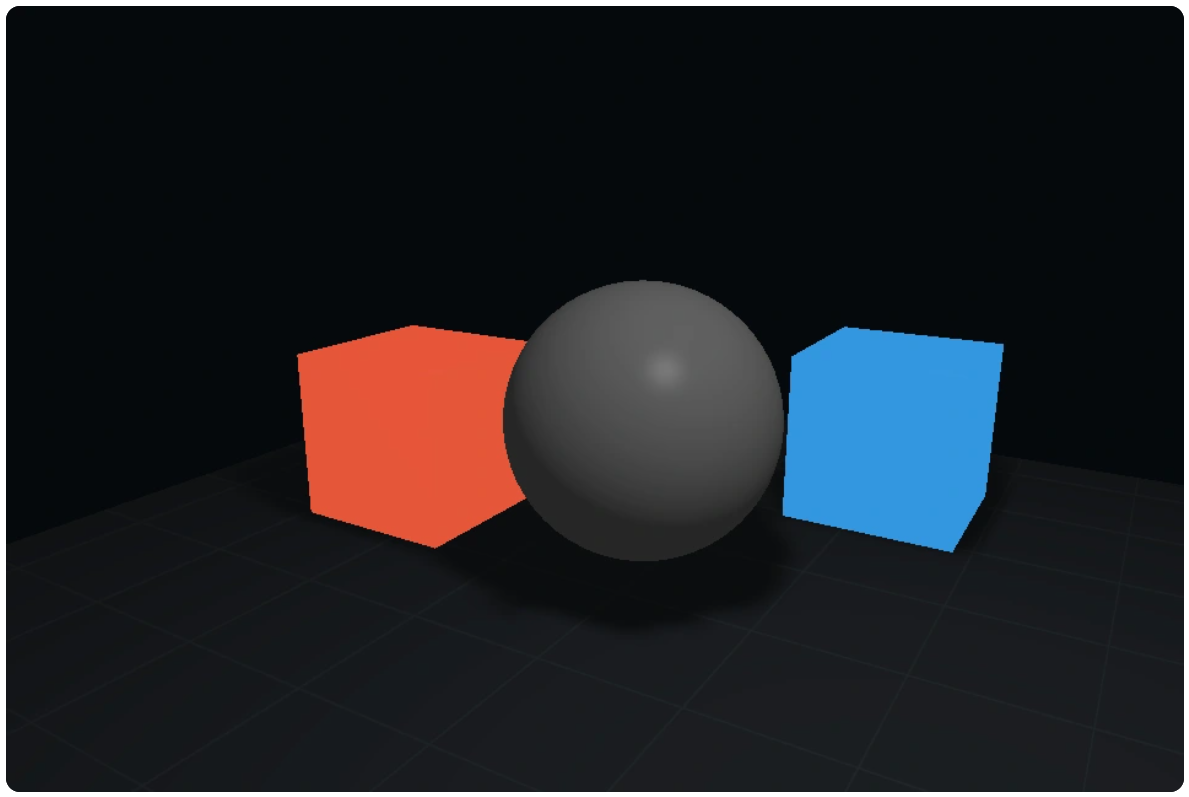
광원의 색·세기와 선택 객체의 Phong / Blinn–Phong, Base color, Emission, Specular, Rim을 Light 탭에서 관리합니다.

03 / SCENE EDITING

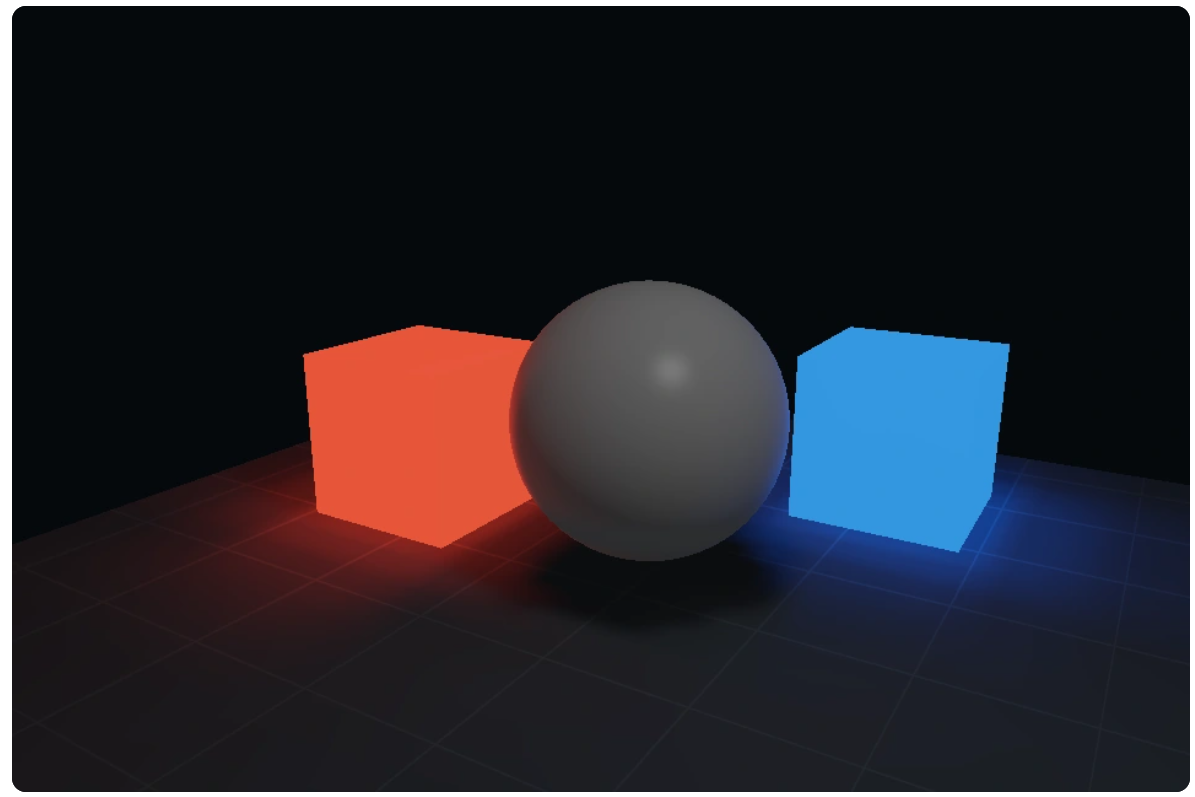
여러 객체로 확인하는 변화

구·큐브·평면의 추가, 복제, 삭제와 개별 재질 설정을 지원합니다. 여러 표면에 빛이 어떻게 반응하는지 함께 확인합니다.

간접광을 관찰하는 Voxel GI



01 GI OFF · 직접광 중심



02 GI ON · 주변 표면으로 전달되는 색

장면 변경 감지 → 64^3 복셀 생성 → 3D 텍스처 + mip → 5방향 Cone Tracing → 직접광 + 간접광

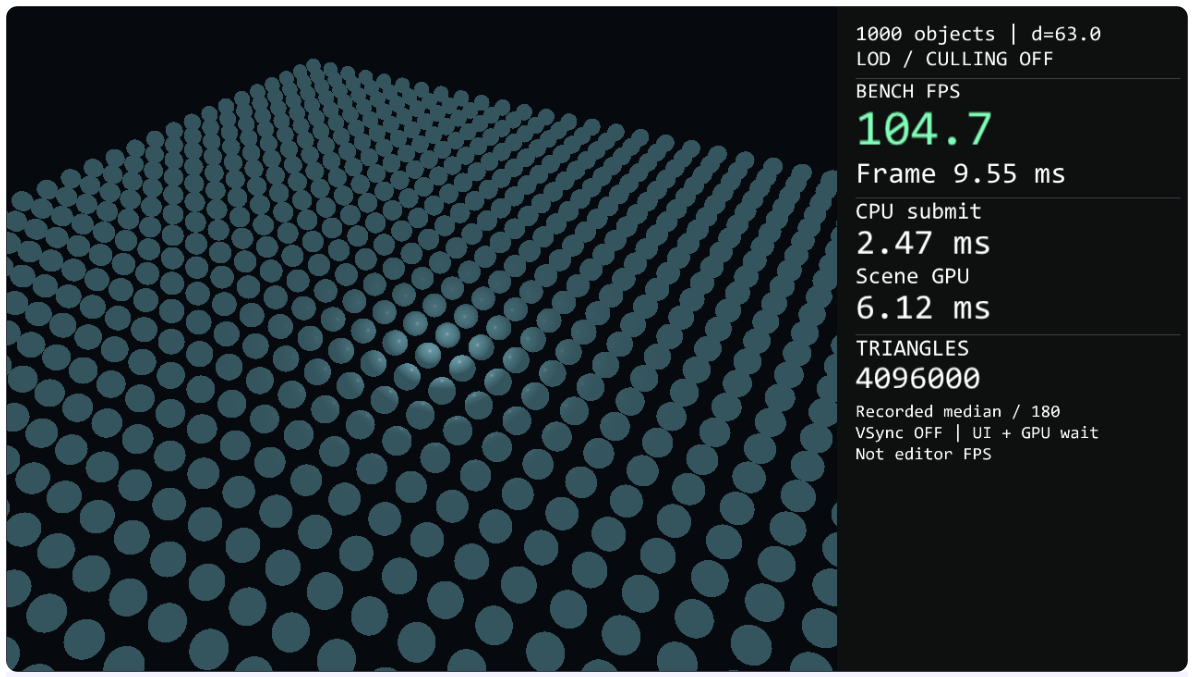
구현 범위

Lumen의 목적 중 동적 간접광을 작은 실험으로 다룹니다. 고정 범위의 해석적 도형 복셀화와 단일 바운스 근사이며, Lumen의 Surface Cache·하드웨어 레이 트레이싱·시간적 누적은 구현하지 않았습니다.

객체 수를 늘리며 확인한 성능 변화

화면 속 통계로 확인하는 LOD·컬링 OFF / ON 비교 · 카메라 거리 63, 32열 격자와 간격 3 고정

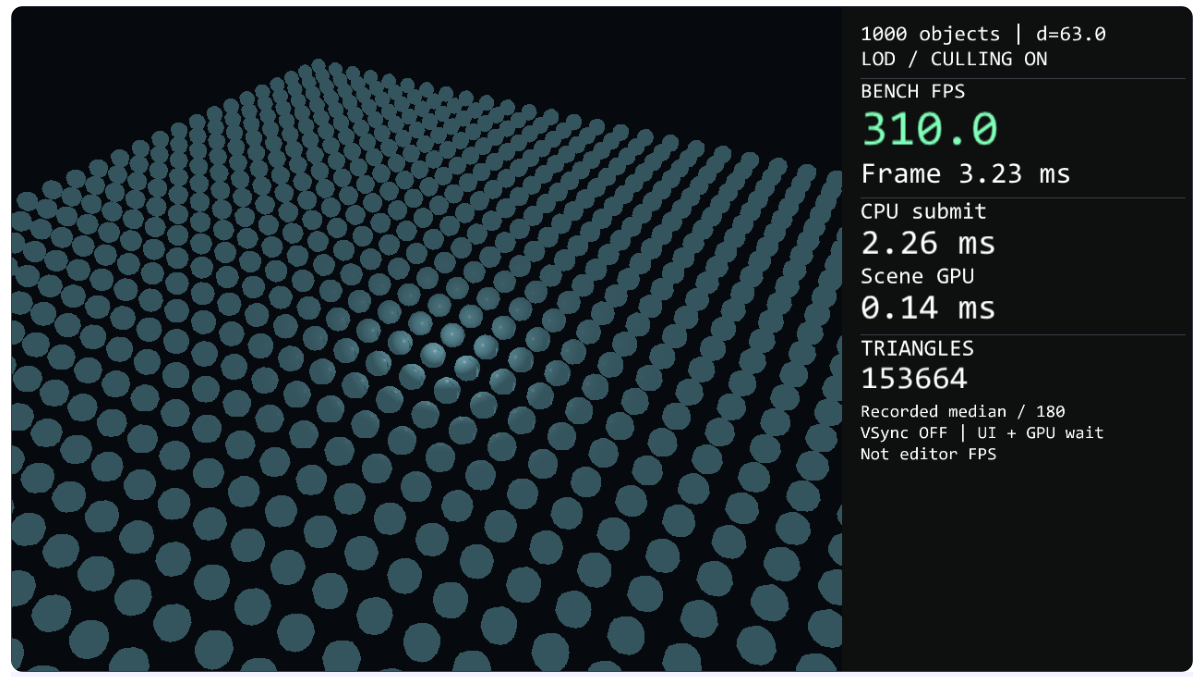
1,000개 · LOD / 컬링 OFF



104.7 FPS **9.55** ms/frame

CPU 2.47 ms · GPU 6.12 ms

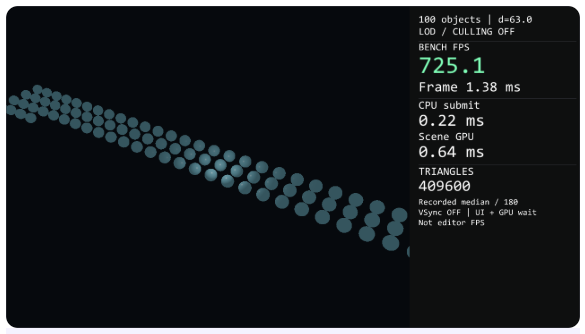
1,000개 · LOD / 컬링 ON



310.0 FPS **3.23** ms/frame

CPU 2.26 ms · GPU 0.14 ms

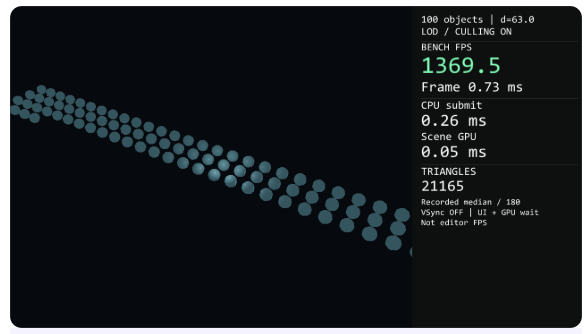
100개 · OFF



725.1 FPS **1.38** ms/frame

CPU 0.22 ms · GPU 0.64 ms

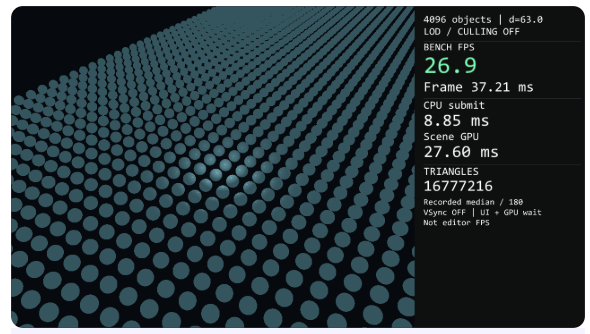
100개 · ON



1369.5 FPS **0.73** ms/frame

CPU 0.26 ms · GPU 0.05 ms

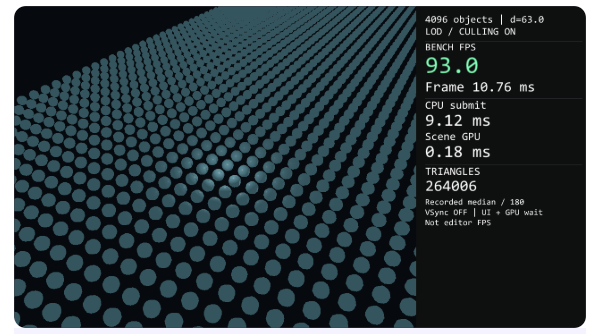
4,096개 · OFF



26.9 FPS **37.21** ms/frame

CPU 8.85 ms · GPU 27.60 ms

4,096개 · ON



93.0 FPS **10.76** ms/frame

CPU 9.12 ms · GPU 0.18 ms

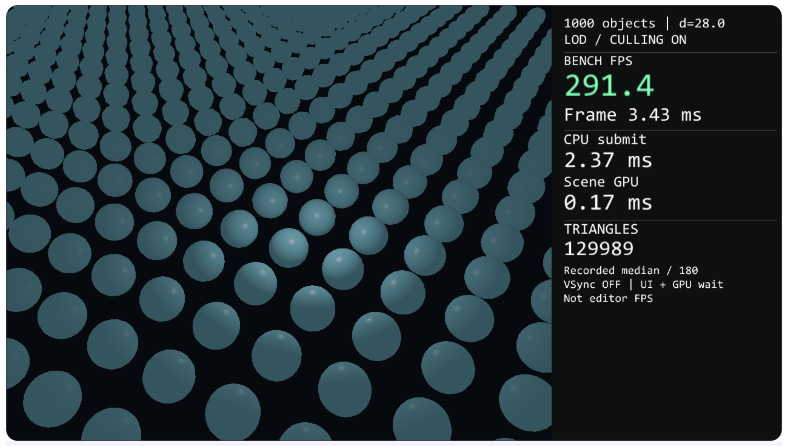
객체 수는 그대로 두고 제출 기하를 줄입니다. CPU 제출 비용과 GPU 시간을 함께 확인하며, 한 수치의 개선을 전체 프레임 성능과 구분합니다. HTML에서는 캡처를 클릭하면 확대됩니다.

별도 측정 창 · Ryzen 7 6800H / AMD Radeon(TM) Graphics · MSVC /O2 · 장면 900×720 + 통계 UI, 창 1280×720
CPU submit = Render 호출 경과 시간(드라이버 대기 포함 가능), Scene GPU = GPU timestamp · Frame = UI·swap·GPU 완료 대기를 포함한 반복 시간
예열 30 + 측정 180프레임 중앙값 · Bench FPS = 1000 / Frame 중앙값, 에디터 FPS가 아님 · VSync·GI·그림자·애니메이션 OFF · 2026.10.21

거리 변화에 따른 LOD와 프레임 비교

상단: 동일한 구 1,000개 장면의 카메라 거리 변경 · LOD와 컬링 ON, LOD error 2 px

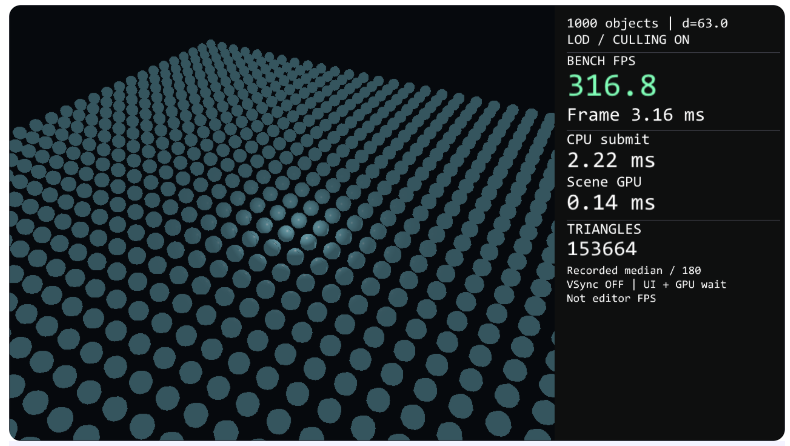
거리 28



291.4 FPS **3.43** ms/frame

CPU 2.37 ms · GPU 0.17 ms

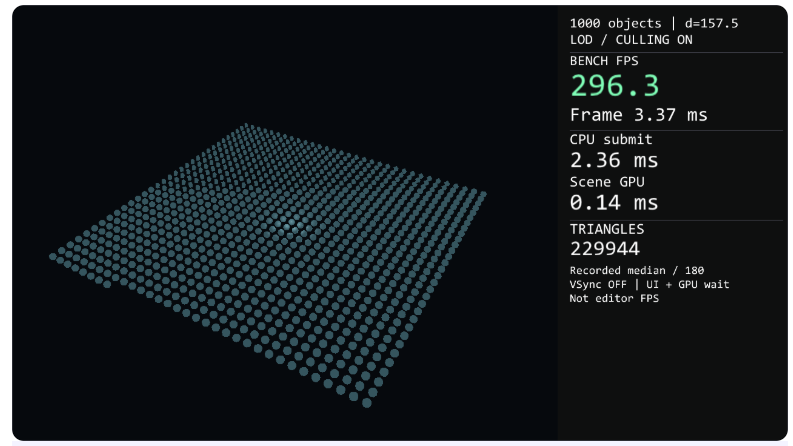
거리 63



316.8 FPS **3.16** ms/frame

CPU 2.22 ms · GPU 0.14 ms

거리 157.5



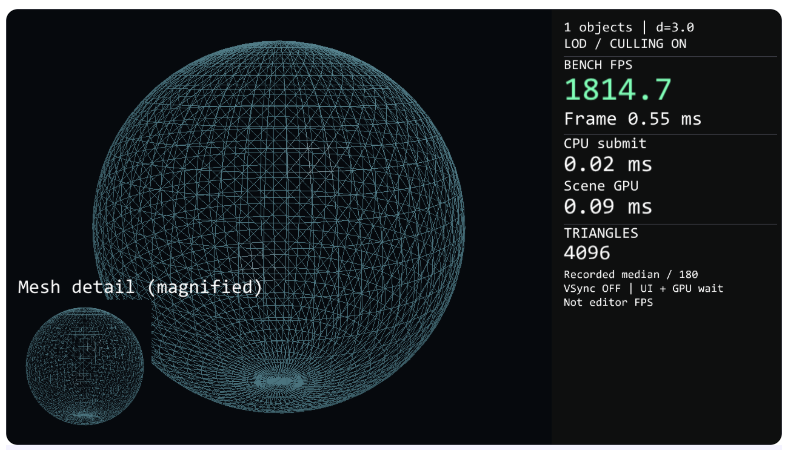
296.3 FPS **3.37** ms/frame

CPU 2.36 ms · GPU 0.14 ms

> 멀어질수록 낮은 LOD가 선택되지만 더 많은 객체가 시야에 들어옵니다. 따라서 거리 증가가 전체 삼각형 수나 프레임 시간의 감소를 보장하지는 않습니다.

하단: 구 하나의 와이어프레임 · 왼쪽 아래는 같은 렌더 결과의 확대 보기

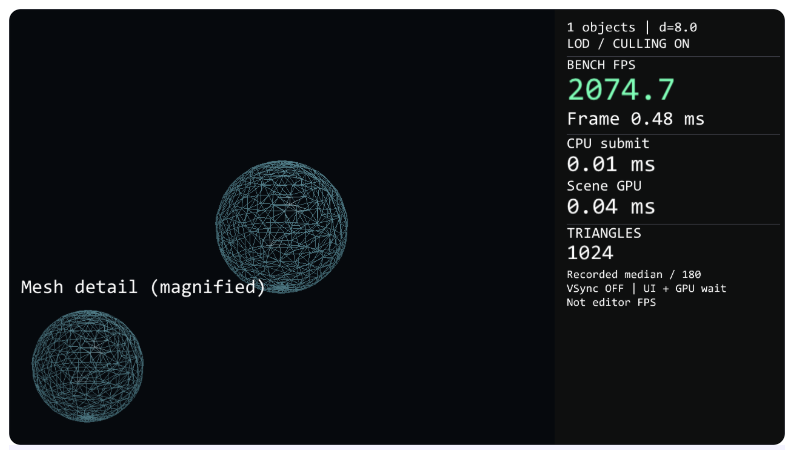
거리 3 · LOD 0 · 4,096 삼각형



1814.7 FPS **0.55** ms/frame

CPU 0.02 ms · GPU 0.09 ms

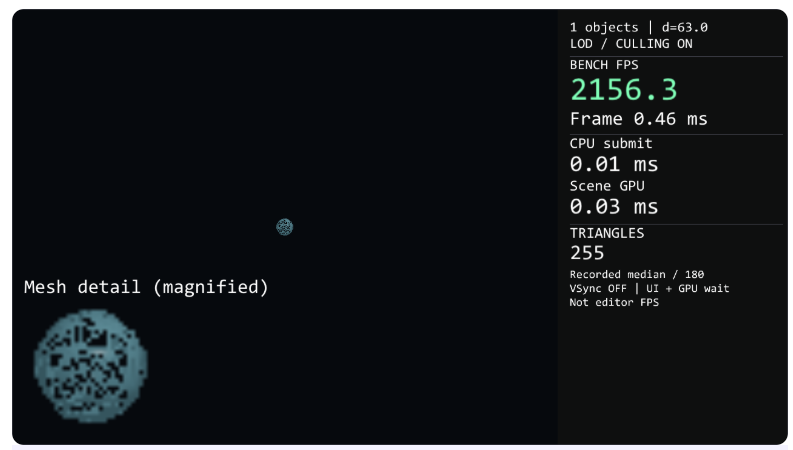
거리 8 · LOD 1 · 1,024 삼각형



2074.7 FPS **0.48** ms/frame

CPU 0.01 ms · GPU 0.04 ms

거리 63 · LOD 2 · 255 삼각형



2156.3 FPS **0.46** ms/frame

CPU 0.01 ms · GPU 0.03 ms

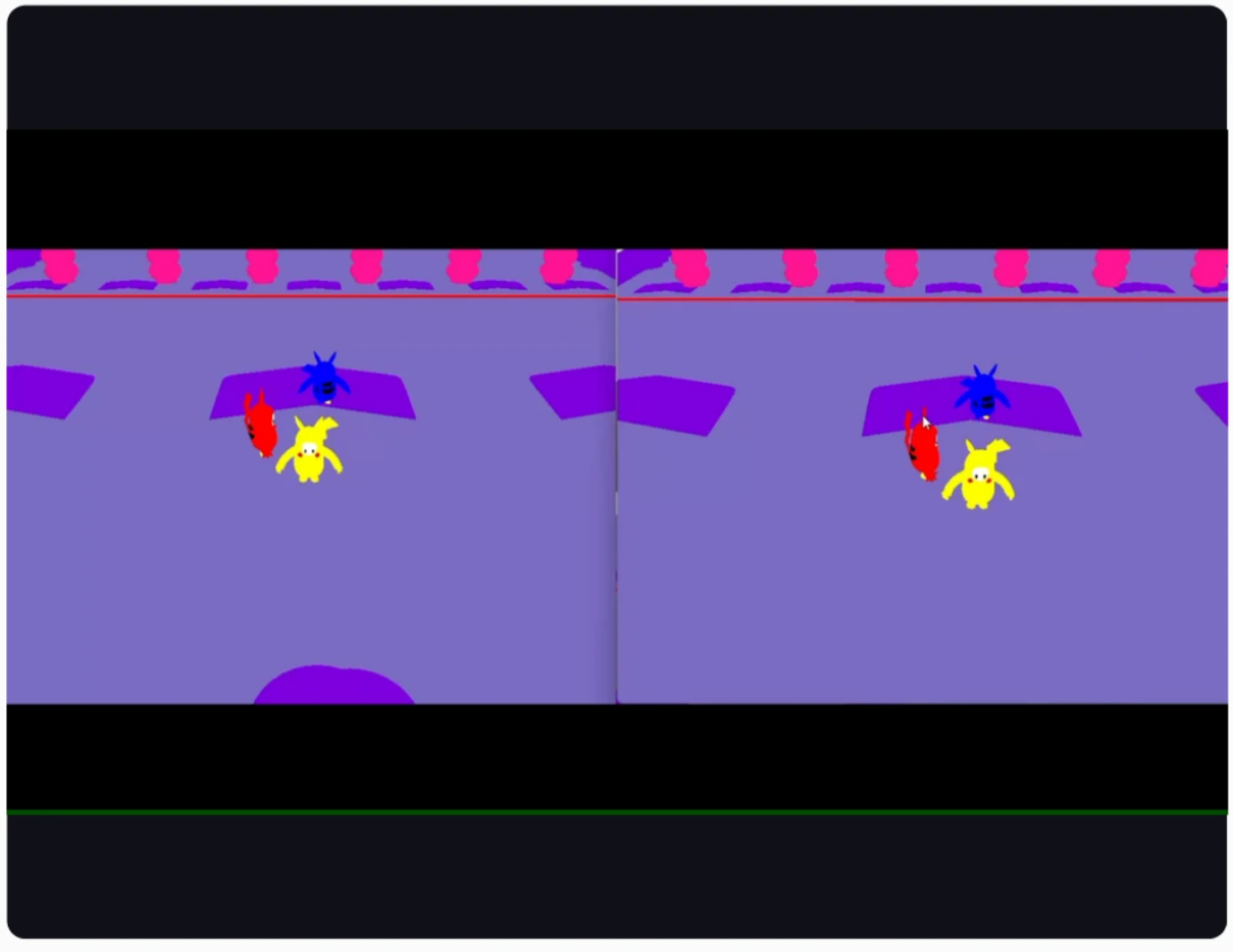
> 4,096 → 1,024 → 255 삼각형으로 메시 밀도가 줄어드는 모습을 확인할 수 있습니다. 와이어프레임은 구조 확인용이며, 상단의 일반 렌더링과 성능을 직접 비교하지 않습니다.

별도 측정 창 · Ryzen 7 6800H / AMD Radeon(TM) Graphics · MSVC /O2 · 장면 900×720 + 통계 UI, 창 1280×720
CPU submit = Render 호출 경과 시간(드라이버 대기 포함 가능), Scene GPU = GPU timestamp · Frame = UI·swap·GPU 완료 대기를 포함한 반복 시간
예열 30 + 측정 180프레임 중앙값 · Bench FPS = 1000 / Frame 중앙값, 에디터 FPS가 아님 · VSync·GI·그림자·애니메이션 OFF · 2026.10.21

Fall Guys Network

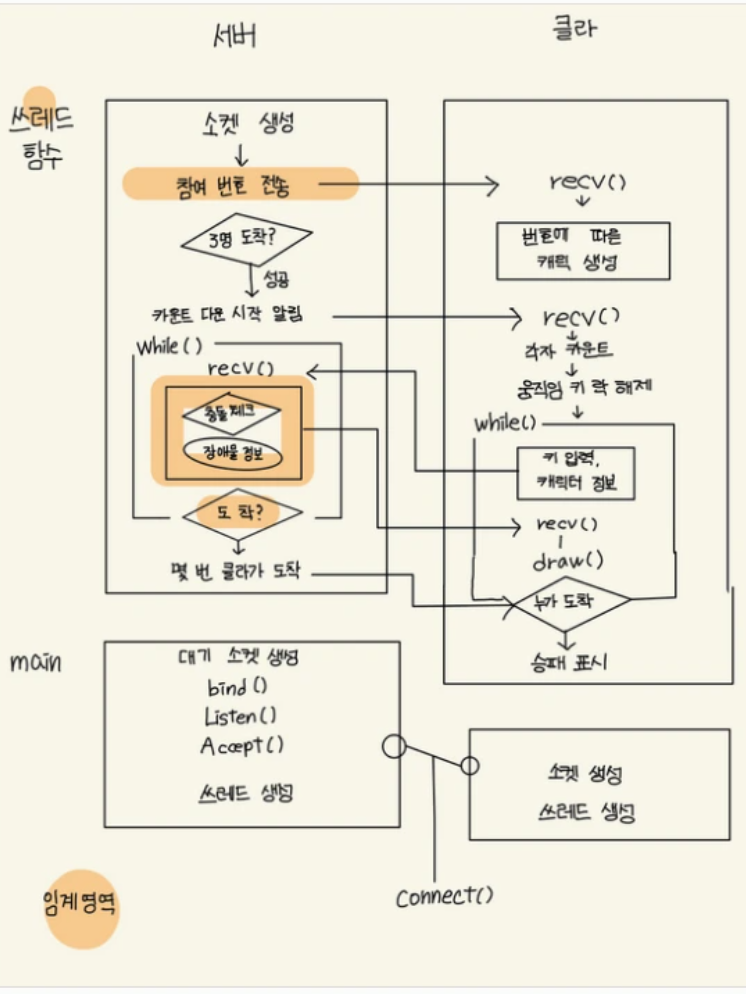
GITHUB

TEAM PROJECT



프로젝트 이름	NGP Fall Guys Network
장르	3D 네트워크 장애물 액션
설명	C++ / OpenGL 로컬 게임을 네트워크 TCP/IP Client–Server 구조로 확장한 팀 프로젝트
개발 인원	3명
담당 역할	네트워크 구조 설계, Character Packet 송수신, 접속 순서 기반 번호 부여, AABB 충돌 판정
개발 언어	C++
사용 IDE	Visual Studio 2022
기술	OpenGL · WinSock TCP/IP · GitHub
제작 기간	2025.11.2 ~ 2025.12.5

네트워크 구조 설계



DESIGN GOAL

Low Level Design

01 접속 및 식별

Accept 순서에 따라 캐릭터 번호를 부여하고 클라이언트별 스레드를 생성

02 C2S 상태 수집

클라이언트가 입력과 Character 상태를 서버로 전송

03 서버 판정

서버에서 캐릭터·장애물 AABB Collision과 도착 상태를 확인

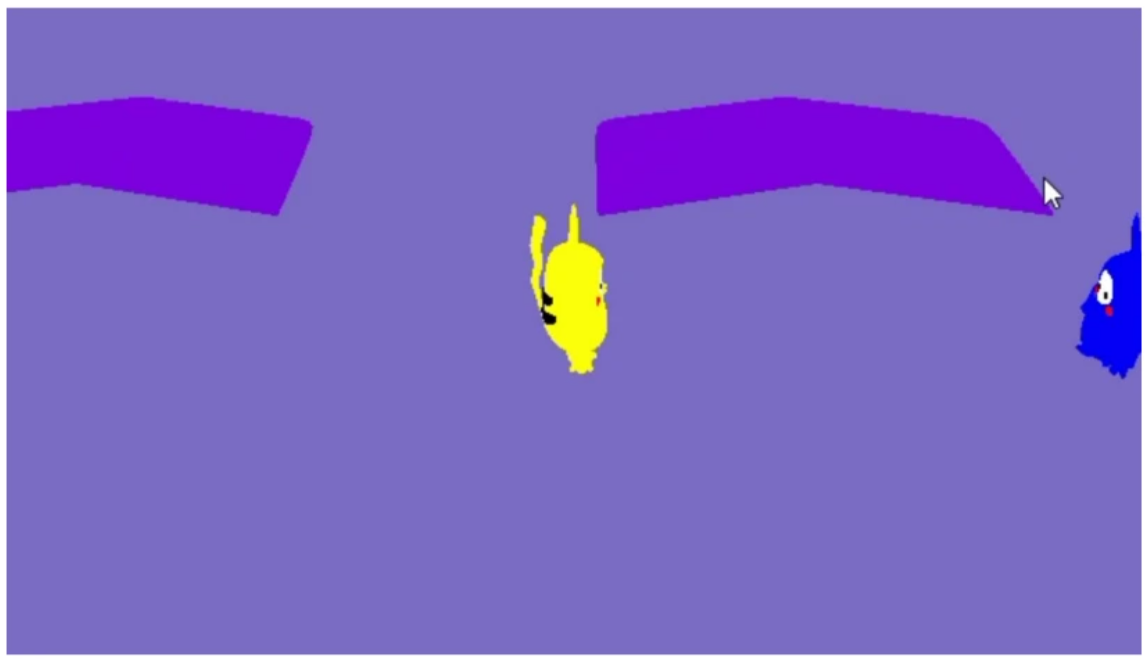
04 S2C 상태 전파

갱신된 상태를 각 클라이언트에 전달해 동일한 장면을 렌더링

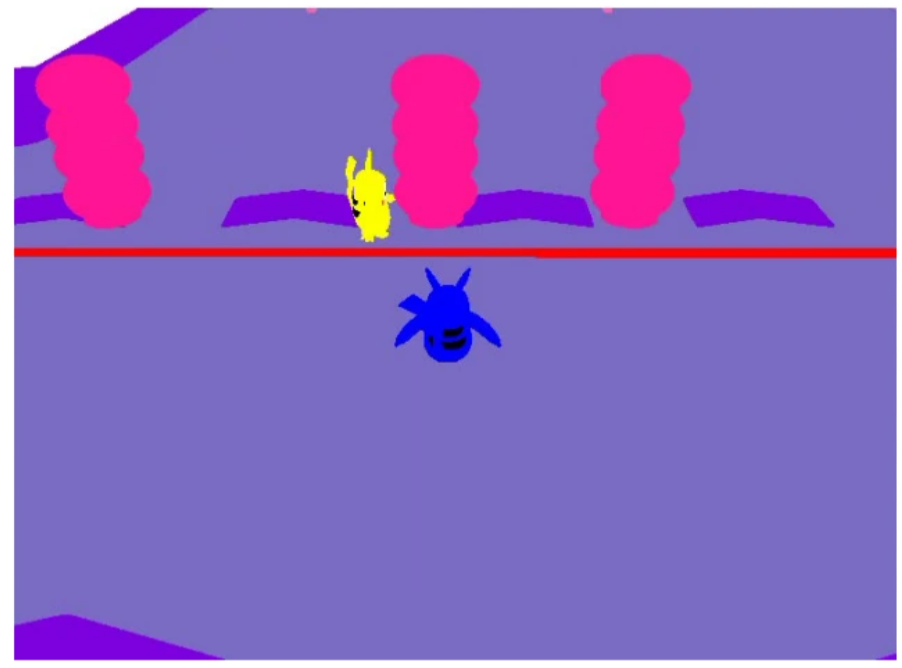
Connect() — Accept() + Player ID — C2S_CHARACTER — Server Update — S2C_CHARACTER

이 프로젝트에서 맡은 역할

플레이어간 동기화



플레이어, 장애물 동기화



GITHUB VIDEO GAME PLAY

17

Team Project

FlickDom (AI VIBE CODING)

14

프로젝트 이름	FlickDom (AI VIBE CODING)
장르	1 VS 1 전략 플릭 보드게임
설명	디스크를 튕겨 보드 칸을 차지하고 카드 패턴을 완성해 점수를 겨루는 물리 기반 파티 게임
개발 인원	팀 프로젝트
담당 역할	Unity MCP 기반 구현 반복, 멀티플레이 흐름, UI/UX, 사운드 제작 및 적용
개발 언어	C#
사용 IDE	Unity 6 Editor, VS Code, Blender, Substance 3D Designer
AI 모델	GPT-5 Codex, Unity MCP, Blender MCP, Substance MCP, VARCO AI Sound
제작 기간	2026.07.01 ~ 2026.08.10

게임 시스템

목표

3개의 디스크로 보드 칸을 점령하고
카드 패턴을 완성합니다.

조작

디스크를 선택한 뒤 드래그 방향과
힘으로 발사해 다음 턴 전략을 만듭니다.

멀티플레이

Host가 방을 생성하고 Join Code를
공유하면 Client가 접속합니다.

시작 전 준비

Host / Client 역할 확인

Join Code 입력

초대를 코드로 교환 및 접속

상태 표시

점수 및 역할 상태 확인

결과 전파

최신 보드 공유 후 시작

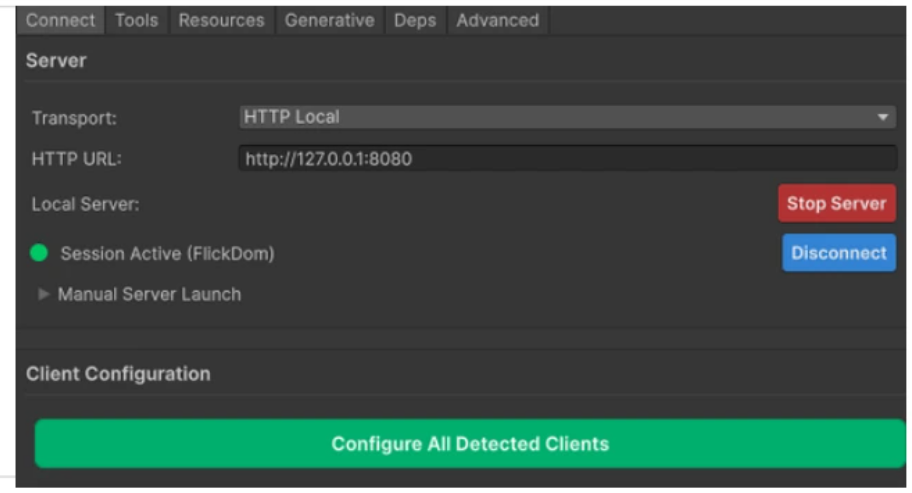
AI/MCP 활용 제작 과정

Unity MCP, Blender MCP, Substance MCP와 AI 사운드 제작 흐름을 함께 활용해
리소스 정리와 구현 반복 속도를 높였습니다.

AI/MCP 기반 개발

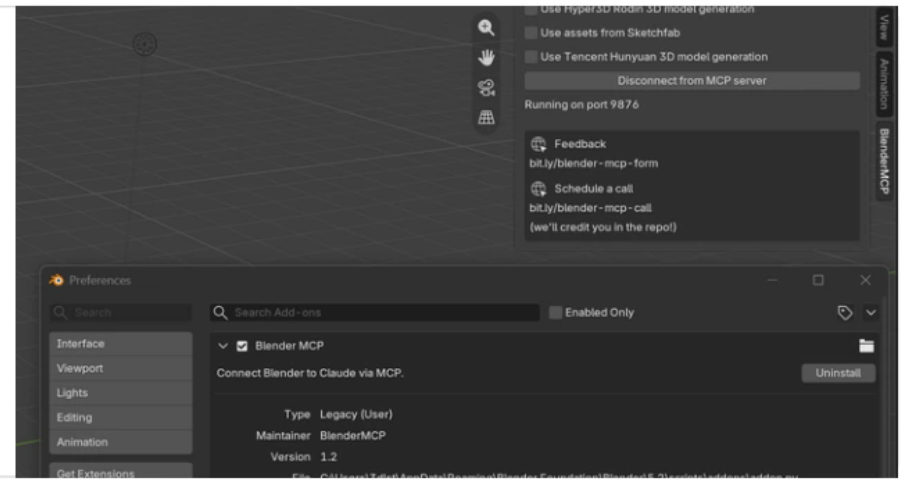
Unity MCP

Unity Editor와 AI 에이전트를 연결해 씬, 오브젝트, 컴포넌트 작업을 빠르게 확인하고 반복했습니다.



Blender MCP

보드, 디스크, 소품 리소스의 비율과 배치 목적을 정리하며 3D 제작 과정을 보조했습니다.



담당 역할

01
Client

보드 규칙과 플레이 흐름 구현

5×5 Board 점령 Logic, Cell 선택·배치 후보 처리, Card Pattern Matching, Stage 진행, Monkey 조작·Camera 흐름, Main Menu를 구현했습니다.

02
Server

턴 FSM과 멀티플레이 상태 전환

초기 Host/Client 동기화 기반 위에서 FSM 기반 턴 흐름과 멀티플레이 상태 전환을 연결하고 Network 구조를 수정했습니다.

03
Build

WebGL 배포와 브라우저 검증

WebGL Build·배포 환경을 구성하고 실제 브라우저에서 접속, 게임 진행, 상태 동기화를 검증해 최종 Release까지 마무리했습니다.

읽어주셔서
감사합니다.